World Wide Web
and
HTTP

OBJECTIVES:

- To discuss the architecture of WWW and describe the concepts of hypertext and hypermedia.
- To describe Web clients and Web servers and their components.
- To define URL as a tool to identify a Web server.
- To introduce three different Web documents: static document, dynamic document, and active document.
- To discuss HTTP and its transactions.
- To define and list the fields in a request message.

OBJECTIVES (*continued*):

- **To define non-persistent and persistent connections in HTTP.**
- **To introduce cookies and their applications in HTTP.**
- **To discuss Web caching, its application, and the method used to update the cache.**

ARCHITECTURE

- The WWW today is a distributed client-server service, in which a client using a browser can access a service using a server.
- However, the service provided is distributed over many locations called sites.
- Each site holds one or more documents, referred to as Web pages. Each
- Web page, however, can contain some links to other Web pages in the same or other sites.
- In other words, a Web page can be simple or composite.

Topics Discussed in the Section

- ✓ **Hypertext and Hypermedia**
- ✓ **Web Client (Browser)**
- ✓ **Web Server**
- ✓ **Uniform Resource Locator (URL)**

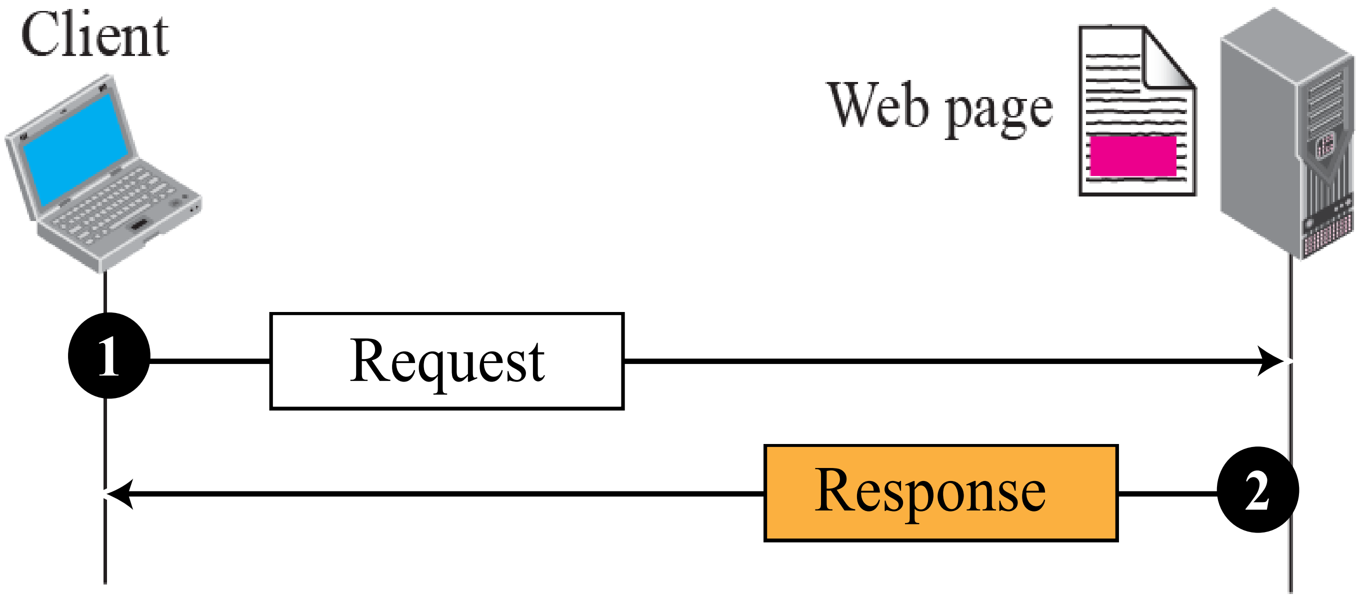
Example

Assume we need to retrieve a Web page that contains the biography of a famous character with some pictures, which are embedded in the page itself.

Since the pictures are not stored as separate files, the whole document is a simple Web page.

It can be retrieved using one single request/ response transaction, as shown in Figure 1.

Figure.1 *Example 22.1*



Example 2

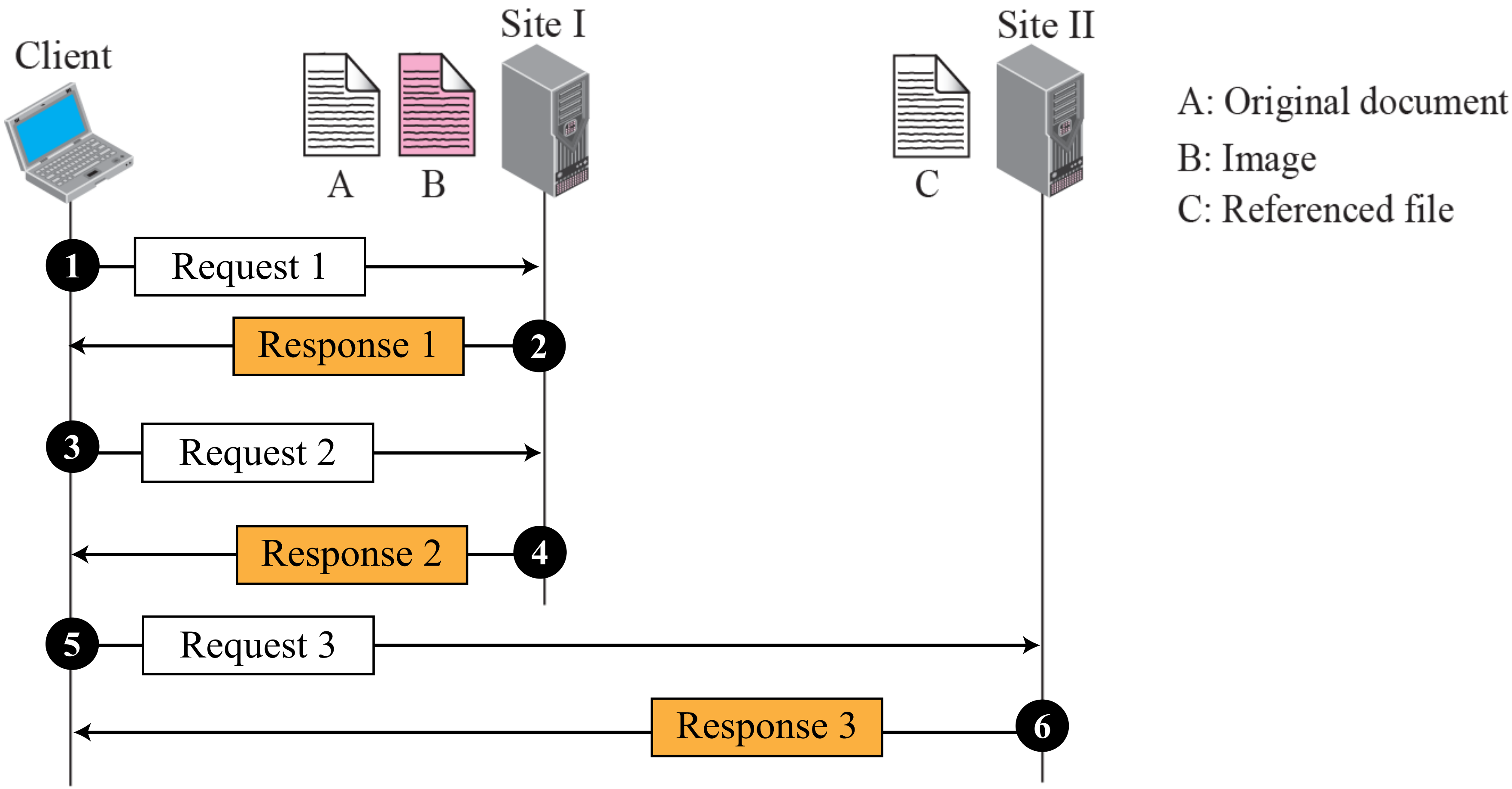
Now assume we need to retrieve a scientific document that contains one reference to another text file and one reference to a large image.

Figure 2 shows the situation. The main document and the image are stored in two separate files in the same site (file A and file B); the referenced text file is stored in another site (file C).

Since we are dealing with three different files, we need three transactions if we want to see the whole document.

The first transaction (request/response) retrieves a copy of the main document (file A), which has a reference (pointer) to the second and the third files.

Figure 2 *Example 22.2*



Example 3

A very important point we need to remember is that file A, file B, and file C in Example 2 are independent Web pages, each with independent names and addresses.

Although references to file B or C are included in file A, it does not mean that each of these files cannot be retrieved independently.

A second user can retrieve file B with one transaction.

A third user can retrieve file C with one transaction.

Figure 3 *Browser*

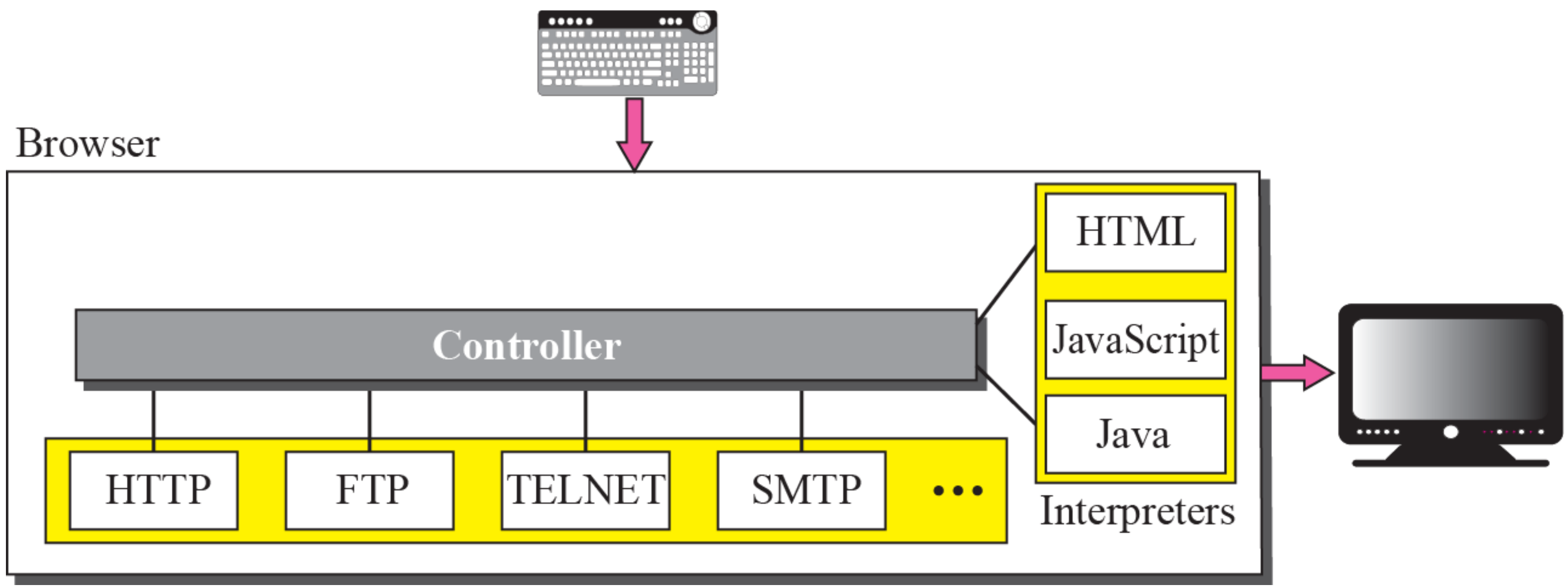
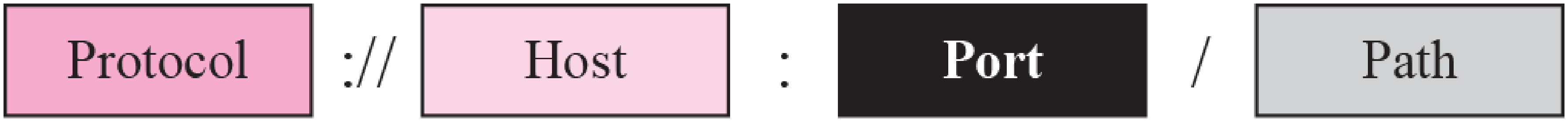


Figure 4 *URL*



WEB DOCUMENTS

The documents in the WWW can be grouped into three broad categories:

1. **Static,**
2. **Dynamic**
3. **Active.**

The category is based on the time the contents of the document are determined.

Topics Discussed in the Section

- ✓ **Static Documents**
- ✓ **Dynamic Documents**
- ✓ **Active Documents**

Figure 5 *Static document*

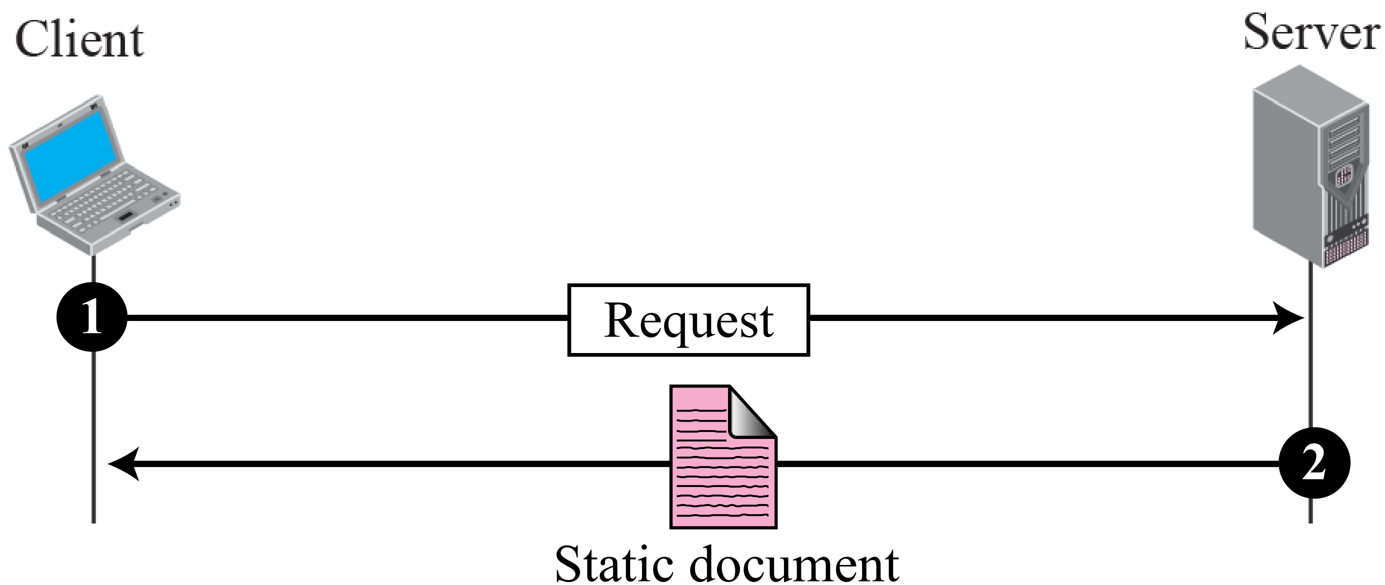


Figure 6 *Dynamic document using CGI*

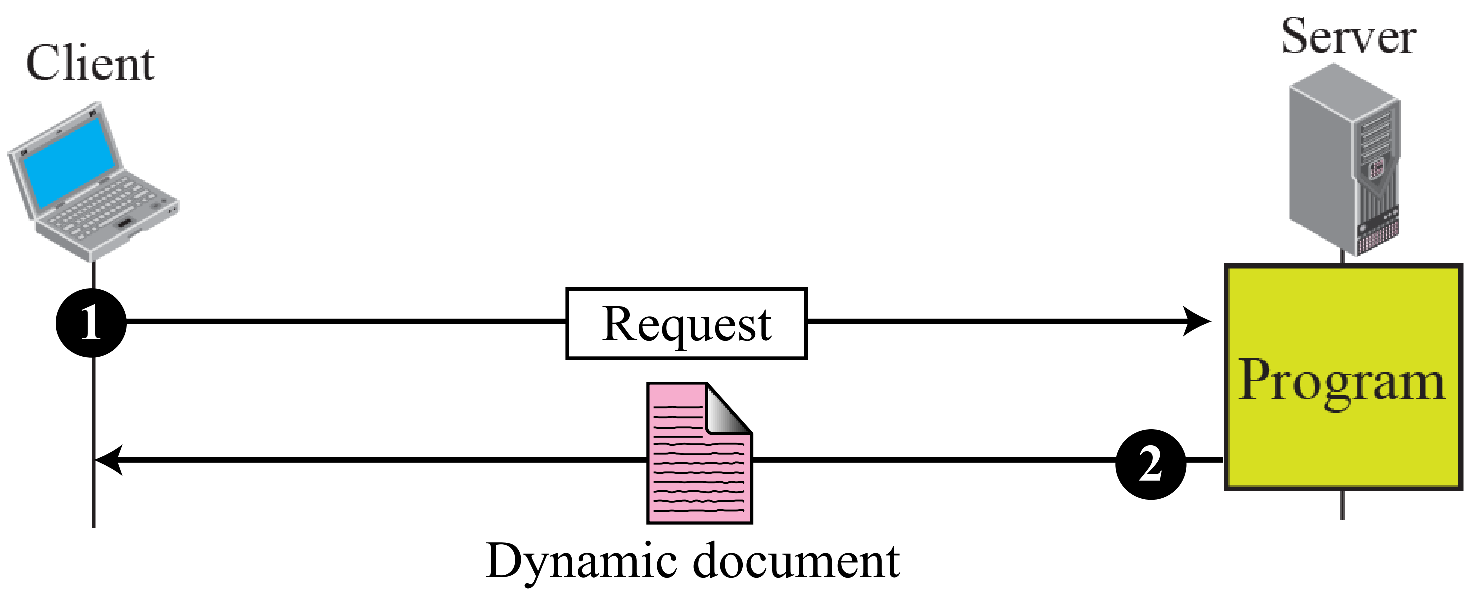
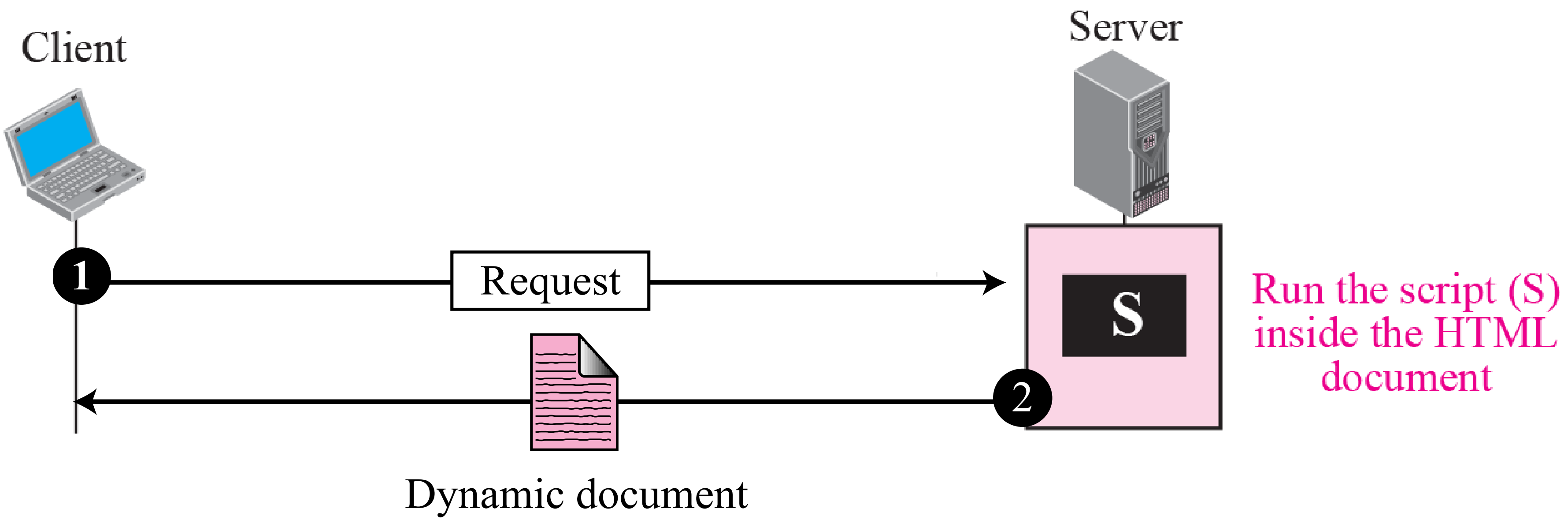


Figure 7 *Dynamic document using server-site script*



Note

Dynamic documents are sometimes referred to as server-site dynamic documents.

Figure 8 *Active document using Java applet*

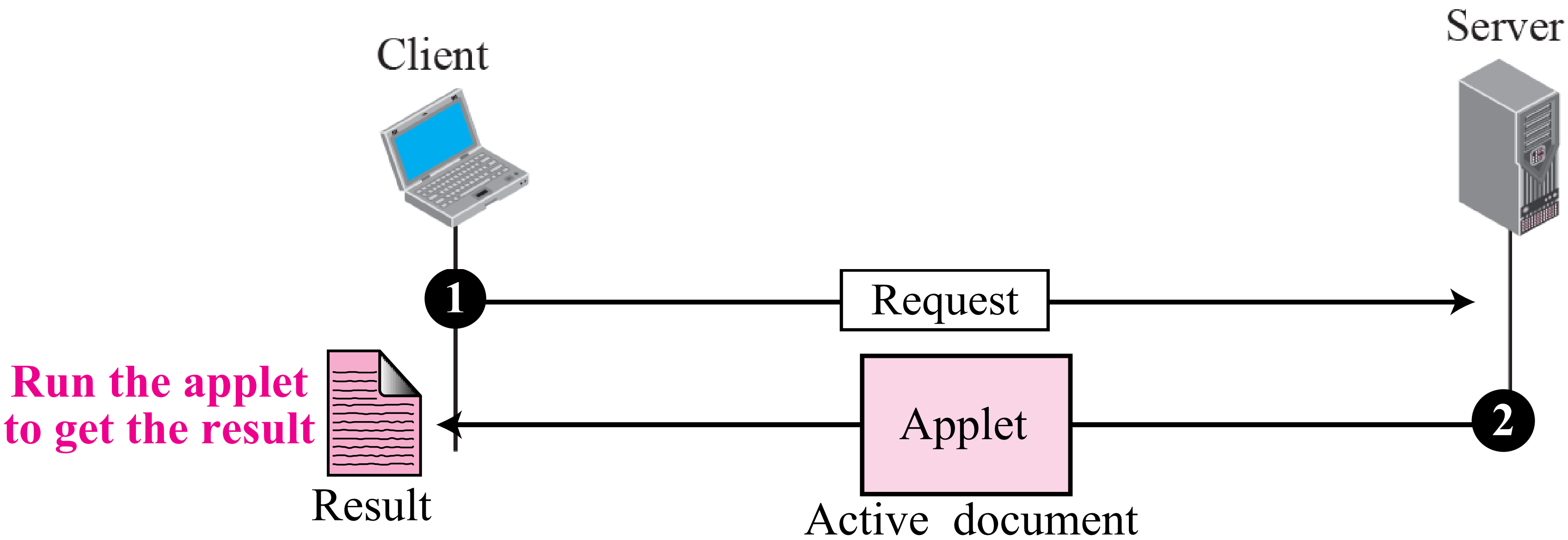
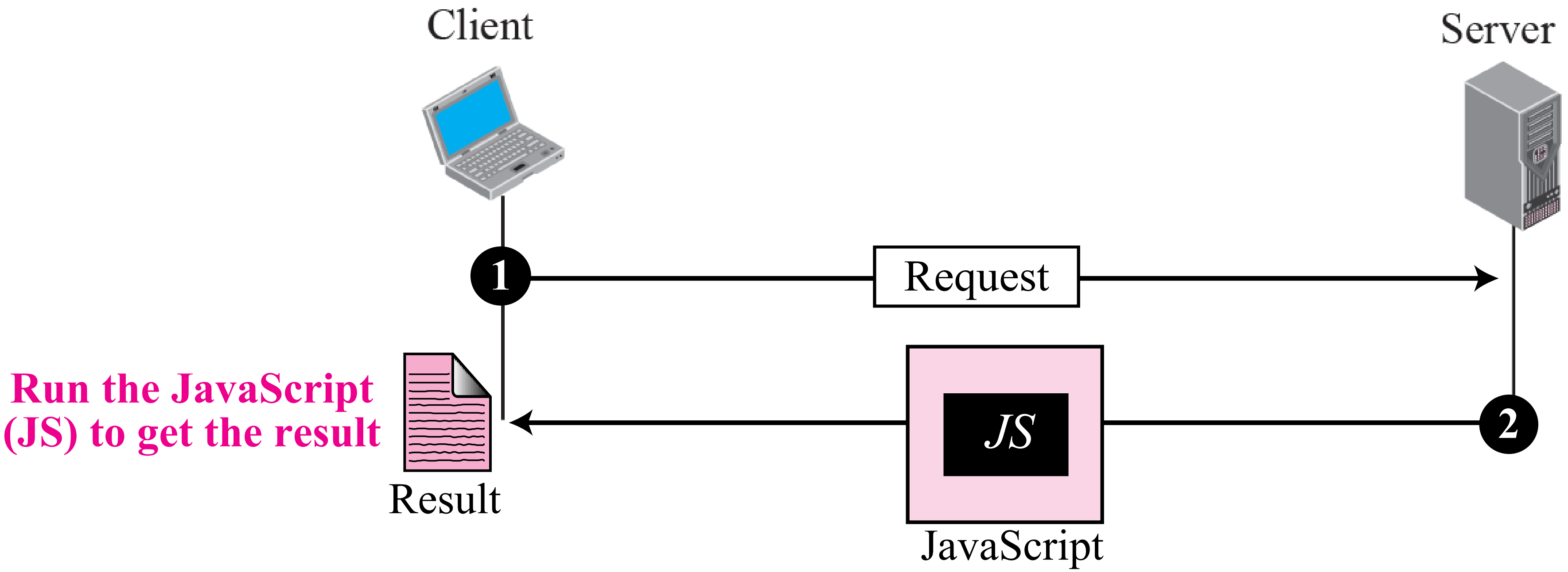


Figure 9 *Active document using client-site script*



Note

Active documents are sometimes referred to as client-site dynamic documents.

HTTP

- The Hypertext Transfer Protocol (HTTP) is a protocol used mainly to access data on the World Wide Web.
- HTTP functions like a combination of FTP and SMTP.
- It is similar to FTP because it transfers files and uses the services of TCP. However, it is much simpler than FTP because it uses only one TCP connection.
- There is no separate control connection; only data are transferred between the client and the server.

Topics Discussed in the Section

- ✓ **HTTP Transaction**
- ✓ **Conditional Request**
- ✓ **Persistence**
- ✓ **Cookies**
- ✓ **Web Caching: Proxy Server**
- ✓ **HTTP Security**

Note

HTTP uses the services of TCP on well-known port 80.

HTTP transaction

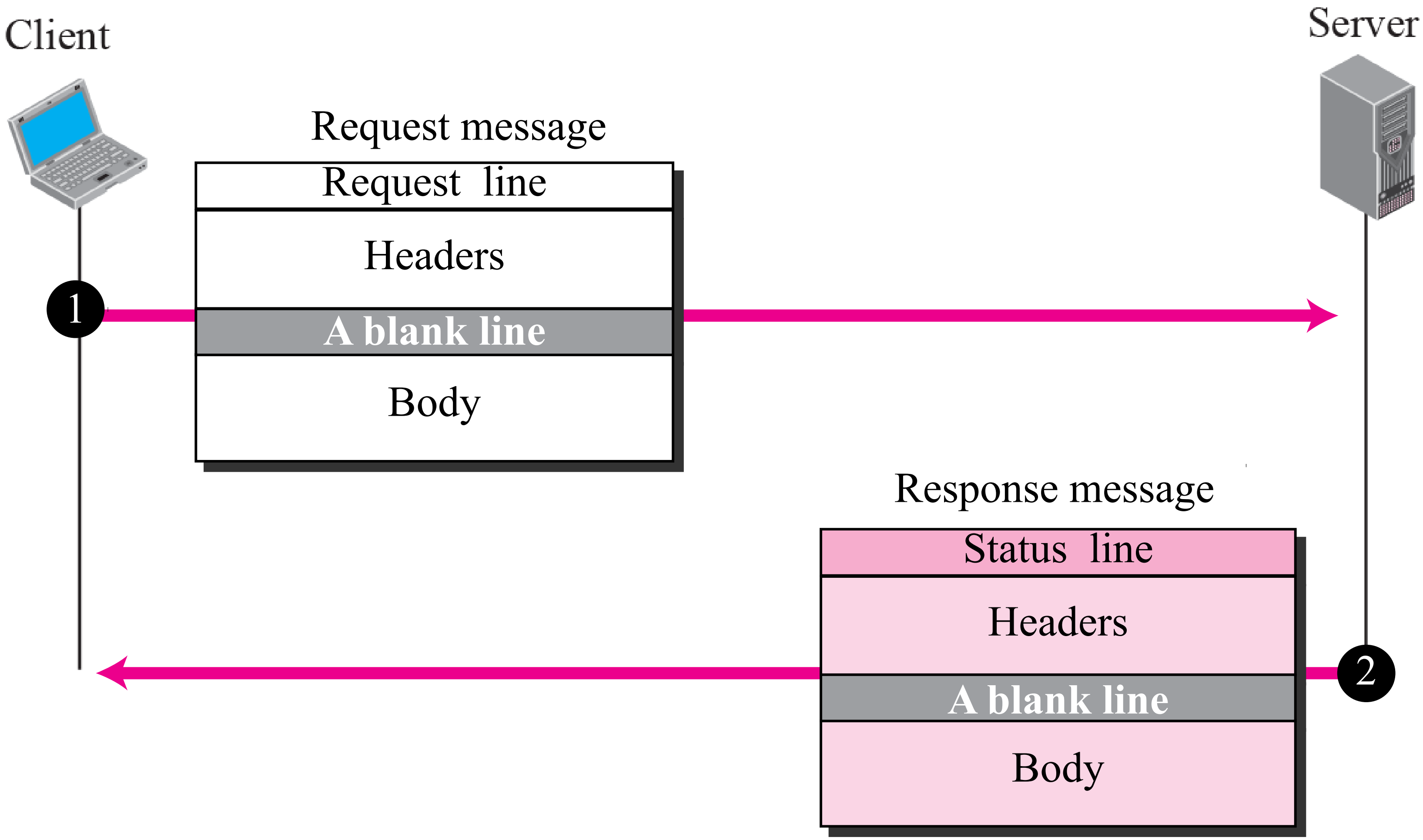


Figure 22.11 *Format of the request message*

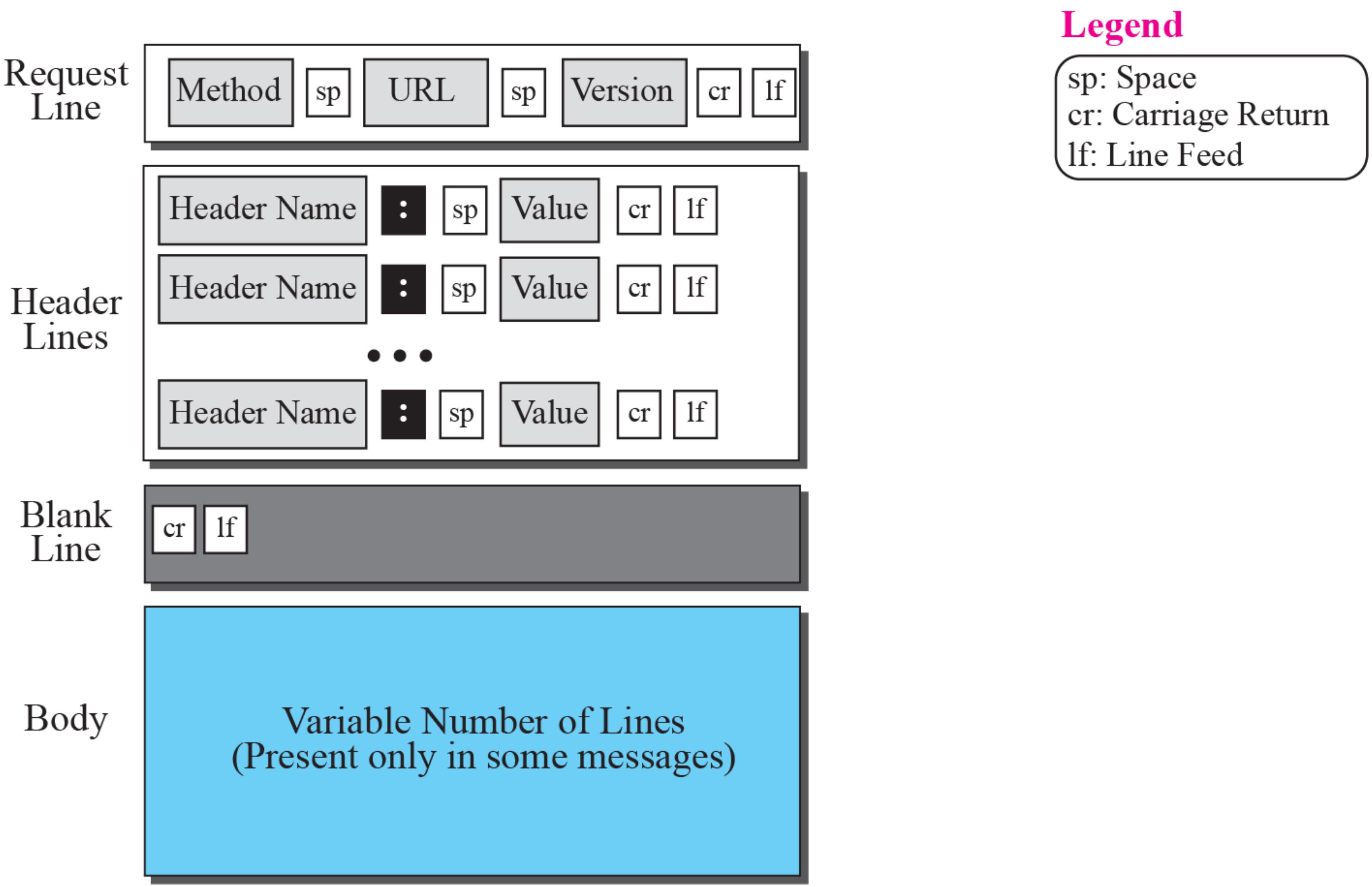


Table 22.1 *Methods*

Method	*Action*
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
POST	Sends some information from the client to the server
PUT	Sends a document from the server to the client
TRACE	Echoes the incoming request
CONNECT	Reserved
DELETE	Remove the Web page
OPTIONs	Enquires about available options

Table 22.2 *Request Header Names*

Header	*Description*
User-agent	Identifies the client program
Accept	Shows the media format the client can accept
Accept-charset	Shows the character set the client can handle
Accept-encoding	Shows the encoding scheme the client can handle
Accept-language	Shows the language the client can accept
Authorization	Shows what permissions the client has
Host	Shows the host and port number of the client
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Cookie	Returns the cookie to the server
If-Modified-Since	Returns the cookie to the server

Figure 12 *Format of the response message*

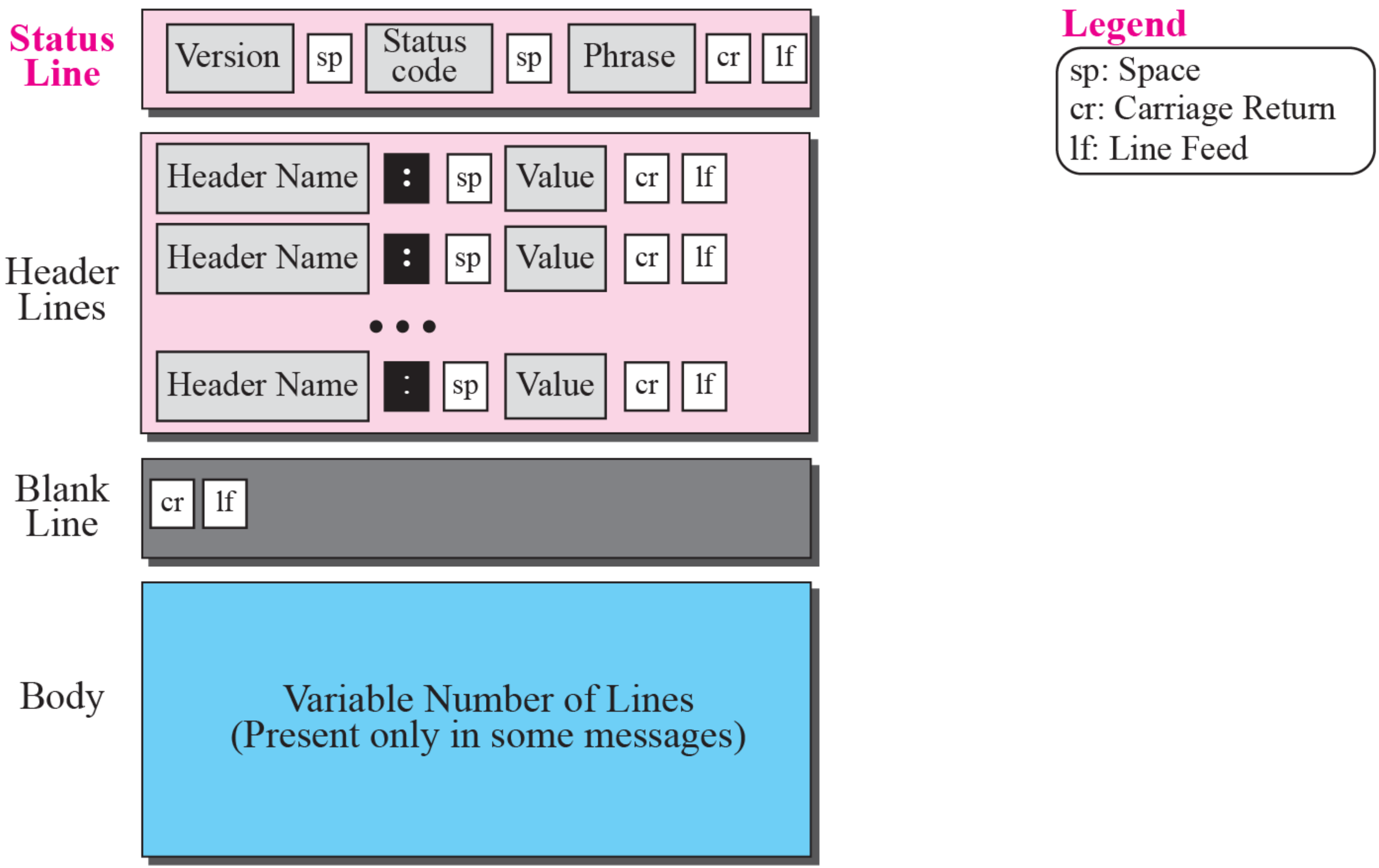


Table 22.3 *Status Codes and Status Phrases*

Status Code	*Status Phrase*	*Description*
	Informational	
100	Continue	The initial part of the request received, continue.
101	Switching	The server is complying to switch protocols.
	Success	
200	OK	The request is successful.
201	Created	A new URL is created.
202	Accepted	The request is accepted, but it is not immediately acted upon.
204	No content	There is no content in the body.
	Redirection	
301	Moved permanently	The requested URL is no longer used by the server.
302	Moved temporarily	The requested URL has moved temporarily.
304	Not modified	The document has not modified.
	Client Error	
400	Bad request	There is a syntax error in the request.
401	Unauthorized	The request lacks proper authorization.
403	Forbidden	Service is denied.
404	Not found	The document is not found.
405	Method not allowed	The method is not supported in this URL.
406	Not acceptable	The format requested is not acceptable.
	Server Error	
500	Internal server error	There is an error, such as a crash, at the server site.
501	Not implemented	The action requested cannot be performed.
503	Service unavailable	The service is temporarily unavailable.

Table 22.4 *Response Header Names*

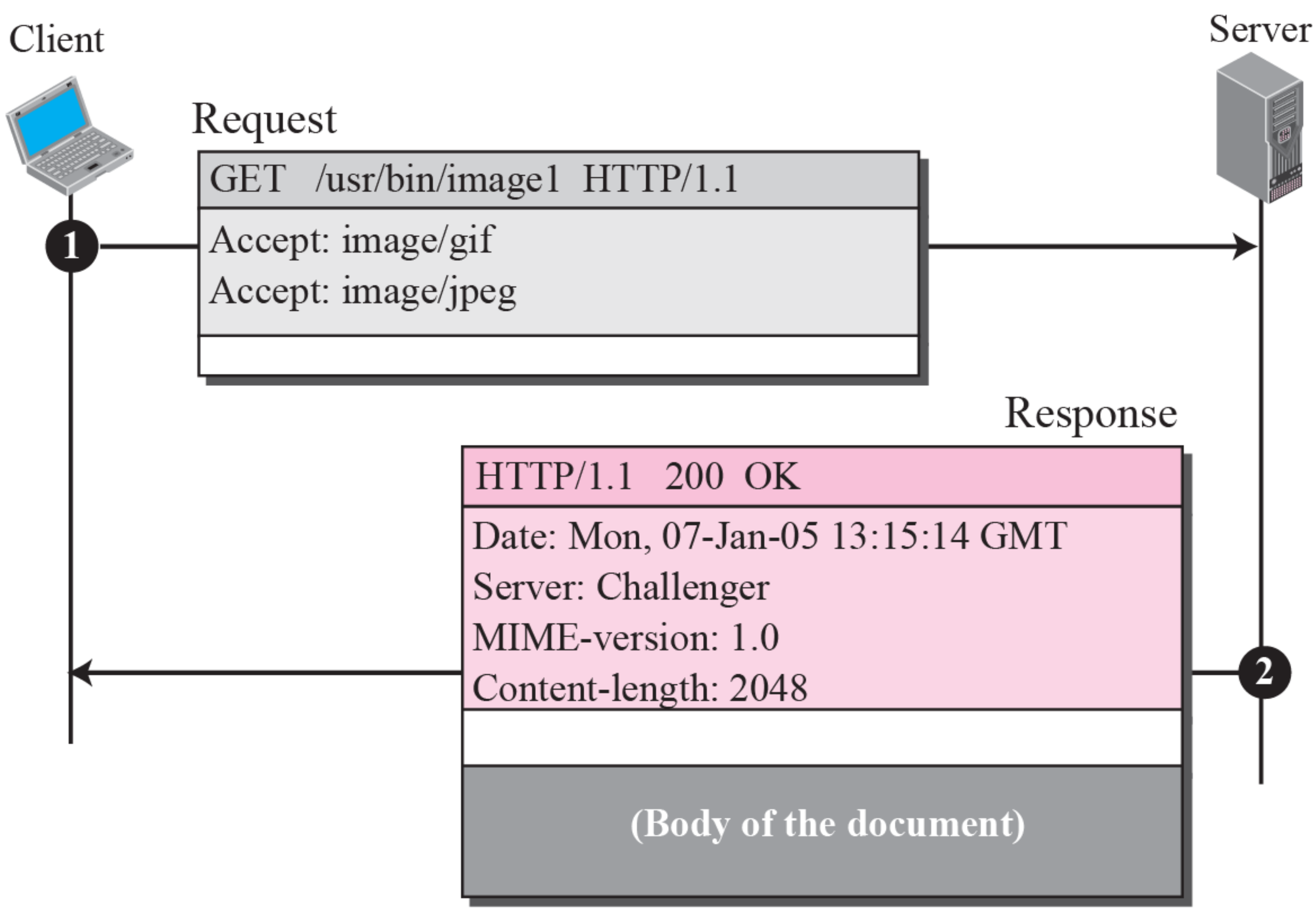
Header	*Description*
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Server	Gives information about the server
Set-Cookie	The server asks the client to save a cookie
Content-Encoding	Specifies the encoding scheme
Content-Language	Specifies the language
Content-Length	Shows the length of the document
Content-Type	Specifies the media type
Location	To ask the client to send the request to another site
Accept-Ranges	The server will accept the requested byte-ranges
Last-modified	Gives the date and time of the last change

Example 4

This example retrieves a document (see Figure 13). We use the GET method to retrieve an image with the path /usr/bin/image1.
The request line shows the method (GET), the URL, and the HTTP version (1.1).
The header has two lines that show that the client can accept images in the GIF or JPEG format.
The request does not have a body.

The response message contains the status line and four lines of header.
The header lines define the date, server, MIME version, and length of the document. The body of the document follows the header.

Figure 22.13 *Example 22.4*

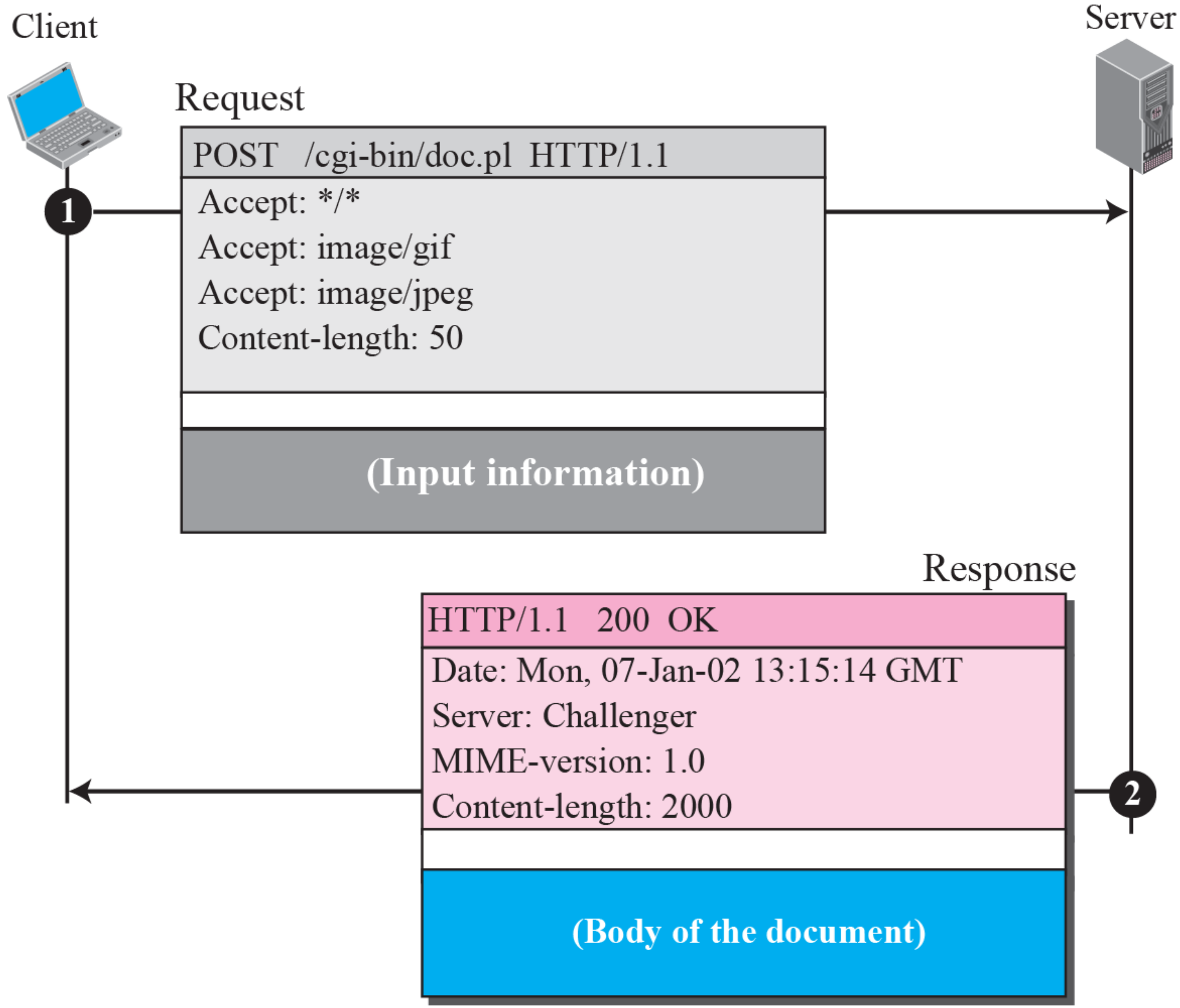


Example 5

In this example, the client wants to send data to the server. We use the POST method.
The request line shows the method (POST), URL, and HTTP version (1.1).
There are four lines of headers. The request body contains the input information.

The response message contains the status line and four lines of headers. The created document, which is a CGI document, is included as the body (see Figure 14).

Figure 14 *Example 5*



Conditional Request

The following shows how a client imposes the modification data and time condition on a request.

```
GET http://www.commonServer.com/information/file1 HTTP/1.1
If-Modified-Since: Thu, Sept 04 00:00:00 GMT
```

The status line in the responds shows the file is not modified after the defined point of time. The body of the response message is also empty.

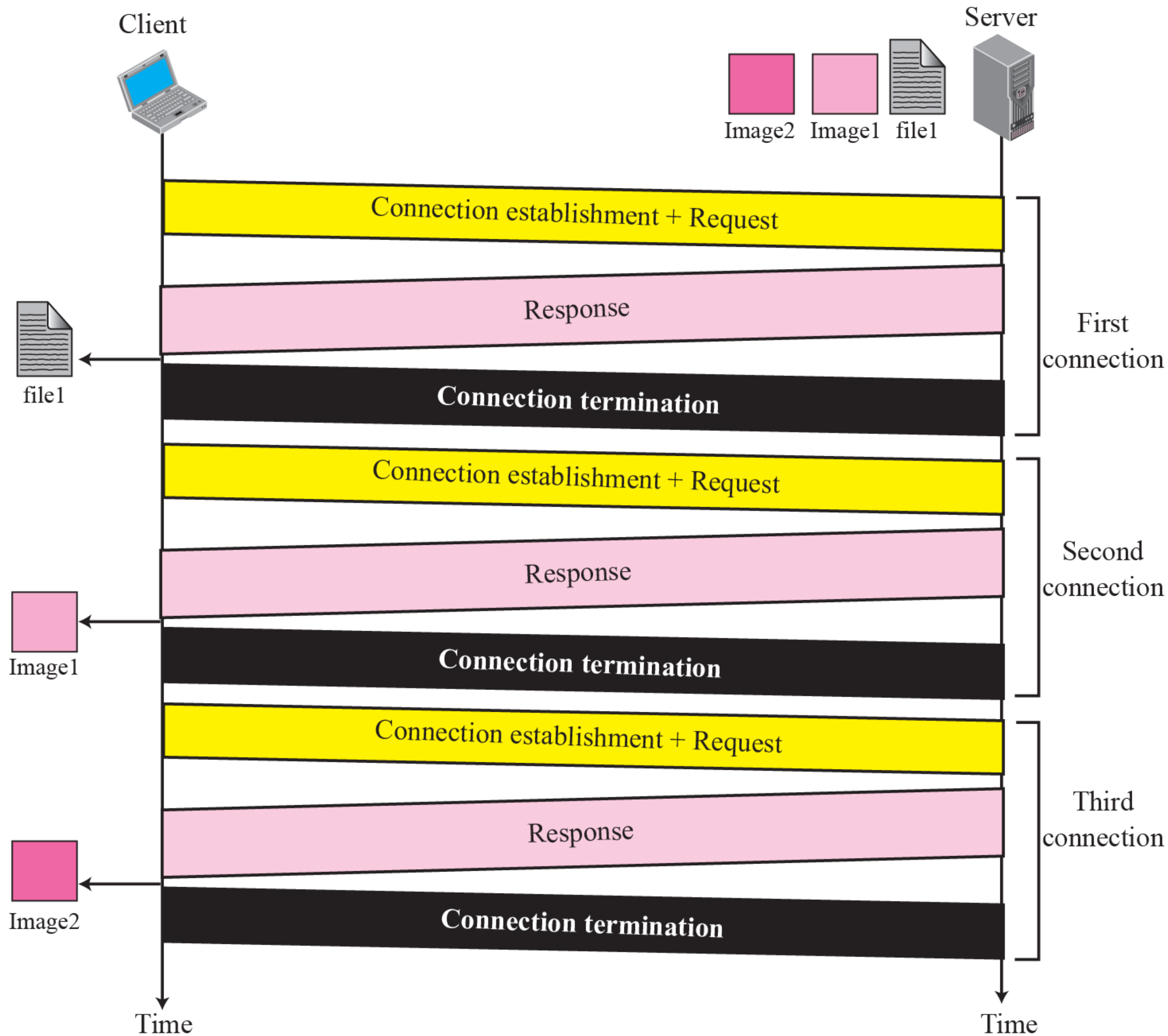
```
HTTP/1.1 304 Not Modified
Date: Sat, Sept 06 08 16:22:46 GMT
Server: commonServer.com

(Empty Body)
```

Nonpersistent connection

- Figure 15 shows an example of a nonpersistent connection.
- The client needs to access a file that contains two links to images.
- The text file and images are located on the same server.

Non-Persistent Connection



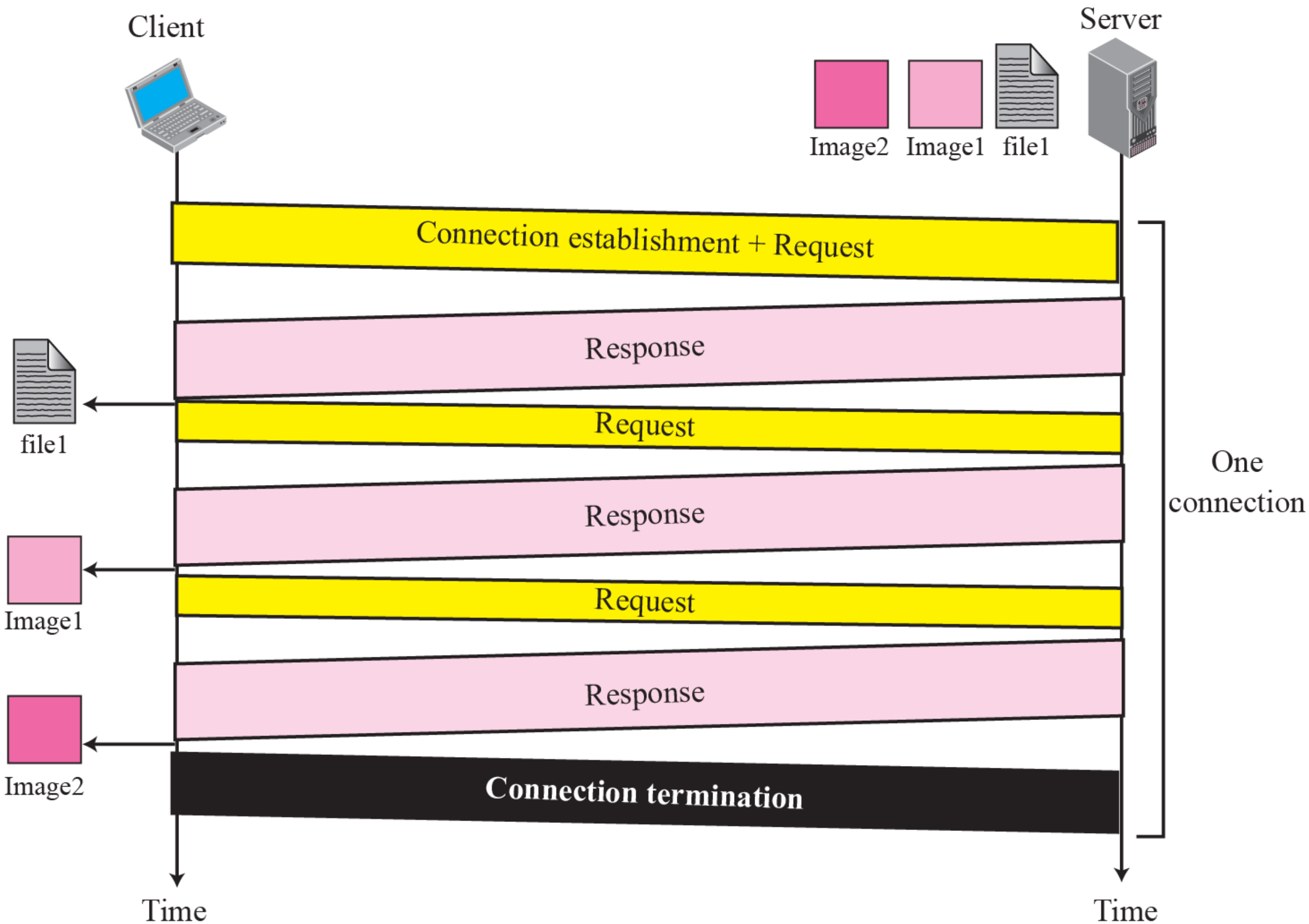
Note

HTTP version 1.1 specifies a persistent connection by default.

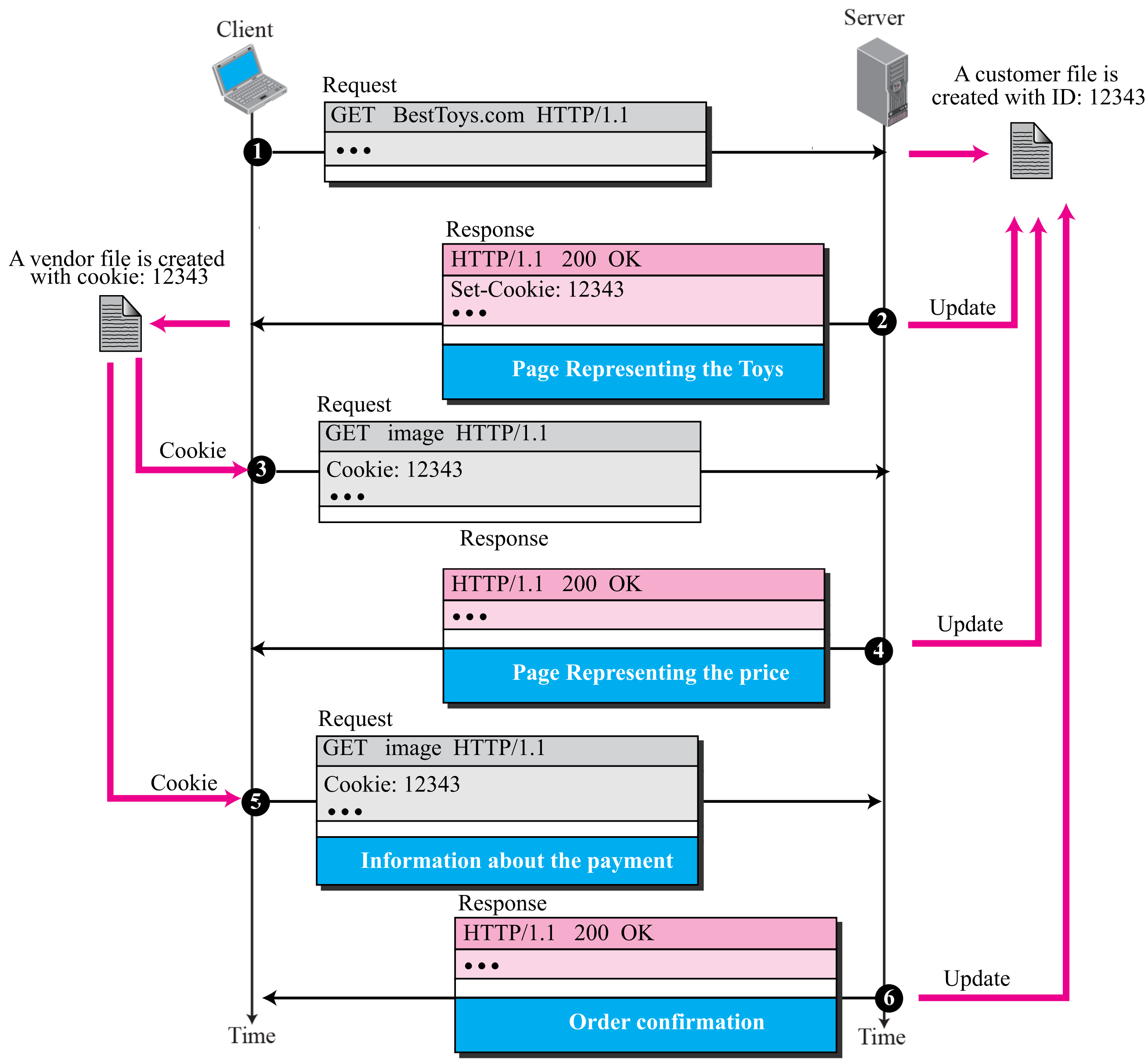
Persistent Connection

Figure 16 shows the same scenario as Example 8, but using persistent connection.

Persistent Connection

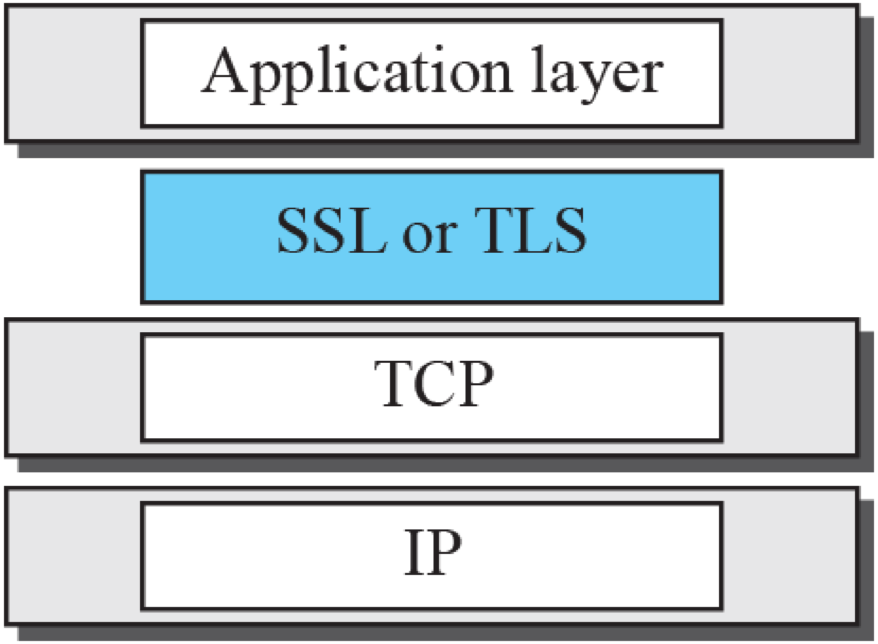


Cookies



Client
Server
A customer file is created with ID: 12343
Request
GET BestToys.com HTTP/1.1
• • •
1
Response
HTTP/1.1 200 OK
Set-Cookie: 12343
• • •
Page Representing the Toys
2
A vendor file is created with cookie: 12343
Update
Request
GET image HTTP/1.1
Cookie: 12343
• • •
Cookie
3
Response
HTTP/1.1 200 OK
• • •
Page Representing the price
4
Update
Request
GET image HTTP/1.1
Cookie: 12343
• • •
Information about the payment
Cookie
5
Response
HTTP/1.1 200 OK
• • •
Order confirmation
6
Update
Time
Time

HTTPS



SSL Services

SSL provides several services on data received from the application layer.

- **Fragmentation.** First, SSL divides the data into blocks of 2^{14} bytes or less.
- **Compression.** Each fragment of data is compressed using one of the lossless compression methods negotiated between the client and server. This service is optional.
- **Message Integrity.** To preserve the integrity of data, SSL uses a keyed-hash function to create a MAC.
- **Confidentiality.** To provide confidentiality, the original data and the MAC are encrypted using symmetric-key cryptography.
- **Framing.** A header is added to the encrypted payload. The payload is then passed to a reliable transport layer protocol.